ಶ್ರೀ ಭಗವಾನ್ ಶ್ರೀಕೃಷ್ಣ ಪರಮಾತ್ಮ

INDEX

NAME: Abhinaya.B.K STD.: SEC.: ROLL NO.: SUB.:

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1)	24/2/26	Basic C programs		
2)	3/3/26	FCFS & SRJF		
3)	10/3/26	Priority scheduling		
4)	17/3/26	Round-Robin scheduling		
5)	24/3/26	Multilevel queue scheduling.		
6)	31/3/26	RMS and EDF		
7)	21/4/26	~~Banker's Alg~~ Bounded Buffer		
		Dining Philosophers		
8)	28/4/26	Banker's Algo		
9)	5/5/25	Dead lock detection		
10)	12/5/25	Memory allocation technique		
11)	26/5/26	Page replacement technique.		

Week-1

1) WAP in C to find the 2nd smallest element in an array

=>
```
#include <stdio.h>
int main(){
    int a[100], n, i, min1, min2;
    printf("Enter size: ");
    scanf("%d", &n);

    for(i=0; i<n; i++)
        scanf("%d", &a[i]);

    min1 = min2 = a[0];
    for(i=1; i<n; i++){
        if(a[i] < min1){
            min2 = min1;
            min1 = a[i];
        } else if(a[i] < min2 && a[i] != min1)
            min2 = a[i];
    }
    printf("Second smallest = %d", min2);
    return 0;
}
```

o/p:
```
Enter size: 5
10 4 6 2 6
second smallest = 4
```

2) C Program to Find Sum of Left Diagonal of a Matrix

```
#include <stdio.h>
int main(){
    int a[10][10], n, i, j, sum = 0;

    printf("Enter order of matrix:");
    scanf("%d", &n);

    for (i=0; i<n; i++)
        for(j=0; j<n; j++)
            scanf("%d", &a[i][j]);

    for (i=0; i<n; i++)
        sum += a[i][i];

    printf("Sum of Left diagonal = %d", sum);

    return 0;
}
```

O/p: Enter order of matrix : 3

1 2 3
4 5 6
7 8 9

sum of left diagonals = 15

FCFS

```
#include <stdio.h>
int main(){
    int n, i, j;
    printf("Enter no of processes:");
    scanf("%d", &n);
    int At[n], Bt[n], CT[n], TAT[n], WT[n], temp;
    printf("Enter Arrival Time and Burst time :\n");

    for(i=0; i<n; i++){
        printf("P %d AT BT:", i+1);
        scanf("%d %d", &At[i], &Bt[i]);
    }
    for (i=0; i<n-1; i++){
        for(j=i+1; j<n; j++){
            if (at[i]>At[j]){
                temp =At[i];
                At[i] = At[j];
                AT[j] = temp;


                temp= BT[i];
                BT[i]= BT[j];
                BT[j]= temp;
            }
        }
    }
```

```
CT[0] = AT[0] + BT[0];
for (i=1; i<n; i++){
    if (CT[i-1] > AT[i]){
        CT[i] = CT[i-1] + BT[i];
    else
        CT[i] = AT[i] + BT[i];
}
for (i=0; i<n; i++){
    TAT[i] = CT[i] - AT[i];
    WT[i] = TAT[i] - BT[i];
}

printf("\n Process \t AT \t BT \t CT \t TAT \t WT \n");
for (i=0; i<n; i++){
    printf("P%d \t %d \t %d \t %d \t %d \t %d \n",
            i+1, AT[i], BT[i], CT[i], TAT[i], WT[i]);
}
return 0;
}
```

O/p:- Enter no. of processes: 4

Enter Arrival and Burst Time:

P_1 AT BT : 0 4

P_2 AT BT : 0 5

P_3 AT BT : 0 2

P_4 AT BT : 0 8

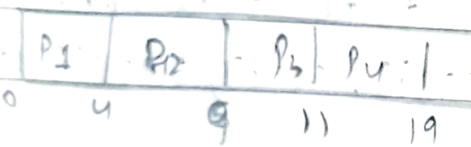
Process	AT	BT	CT	TAT	WT
P_1	0	4	4	4	0
P_2	0	5	9	9	4
P_3	0	2	11	11	9
P_4	0	8	19	19	11
			43	43	24
			4	4	4
			= 10.		

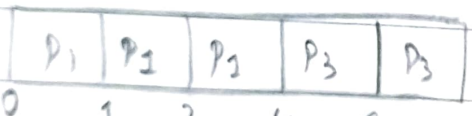
Average CT = 10.74

Average TAT = 10.74

Average WT = 6

	AT	BT
P_1	0	4
P_2	1	8
P_3	2	6
P_4	5	3





at 1 => $P_1 = 3$

$P_2 = 8$

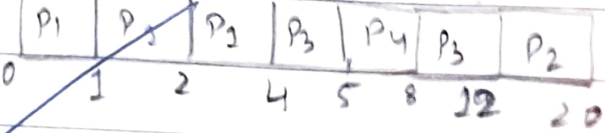
at 2 => $P_1 = 2$

$P_2 = 8$

$P_3 = 6$

at 5 => $P_2 = 8$

$P_3 = 5$



at 1 => $P_1 = 3$

$P_2 = 8$

at 2 => $P_1 = 2$

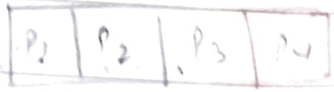
$P_2 = 8$

$P_3 = 6$

at 5 => $P_3 = 4$

$P_2 = 8$

$P_4 = 6\ 3$



SJF:-

```
#include <stdio.h>

int main(){
    int n, i, choice;
    printf("Enter no. of processes :");
    int AT[n], BT[n], CT[n], TAT[n], WT[n], visited[n];

    printf("Enter Arrival Time and Burst Time :\n");
    for(i=0; i<n; i++){
        printf("P%d AT BT ", i+1);
        scanf("%d %d", &AT[i], &BT[i]);
        RT[i]= BT[i];
        visited[i]=0;
    }
    printf("\n Choose Scheduling Type\n");
    printf("1. Non-Preemptive SJF\n");
    printf("2. Preemptive (SRTF)\n");
    scanf("%d", &choice);

    int completed =0, current_time =0, shortest;
    float total_TAT = 0, total_WT = 0;

    if(choice == 1){
        while completed <n){
            shortest = -1;
            for(i=0; i<n; i++){
```

```
        if (AT[i] <= current_time && visited[i] == false)
            if (shortest == -1 || BT[i] <
                    bt[shortest])
                shortest = i;
            }
        }
        if (shortest == -1){
            current_time++;
        }else {
            current_time += bt[shortest];
            CT[shortest] = current_time;
            visited[shortest] = 1;
            completed++;
        }
    }
}else if (choice == 2){
    while (completed < n){
        shortest = -1;
        for (i=0; i<n; i++){
            if (AT[i] <= current_time && rt[i] > 0){
                if (shortest == -1 || rt[i] < rt[shortest])
                    shortest = i;
            }
        }
        if (rt[shortest] == 0){
            ct[shortest] = current_time;
            completed++;
        }
    }
}
```

```
for (i=0; i<n; i++){
    TAT[i] = CT[i] - AT[i];
    WT[i] = TAT[i] - BT[i];

    total_tat += TAT[i];
    total_wt += wt[i];
}
printf("\n Process \t AT \t BT \t CT \t TAT \t WT \n");
for(i=0; i<n; i++){
    printf("\n %d \t %d \t %d \t %d \t %d \t %d \n",
        i+1, AT[i], BT[i], CT[i], TAT[i], WT[i]);
}
printf("Average TAT = %.2f \n", total_TAT/n);
printf("Average WT = %.2f \n", total_wt/n);

return 0;
}
```

O/P:- Enter no. of process : 2
Enter Arrival and Burst time:
P1 AT BT : 0 3
P2 AT BT : 5 8

Average TAT : 5.50
Average WT : 0.00

Choose scheduling type:-
1. Non-Preemptive SJF
2. Preemptive (SRTF)
Enter choice : 1

Process	AT	BT	CT	TAT	WT
P_1	0	3	3	3	0
P_2	5	8	13	8	0

Enter no. of processes: 4

Enter Arrival time and Burst Time:

P_1 AT BT : 0 1

P_2 AT BT : 4 3

P_3 AT BT : 6 2

P_4 AT BT : 1 5

Choose scheduling type:

2: Preemptive SJF (SRTF)

Process	AT	BT	CT	TAT	WT
P_1	0	1	1	1	0
P_2	4	3	11	7	4
P_3	6	2	8	2	0
P_4	1	5	6	5	0

Average TAT = 3.75

Average WT = 1.00

	FCFS	SJF	SRTF
	Avg WT = 4.75	4	4
	Avg TAT = 9.75	9	9
3/3	Avg BT = 4.75	4	3.75

SRTF is more optimal

FCFS takes more time

WEEK-3

Priority (Non-Preemptive)

```
#include <stdio.h>
struct process{
    int pid, bt, pr, ct, wt, tat;
};
int main(){
    struct process p[20], temp;
    int n, i, j;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of process:");
    scanf("%d", &n);

    for(i=0; i<n; i++){
        p[i].pid = i+1;
        printf("Enter Burst Time and Priority for P%d:", i+1);
        scanf("%d %d", &p[i].bt, &p[i].pr);
    }
    for(i=0; i<n-1; i++){
        for(j=i+1; j<n; j++){
            if(p[i].pr > p[j].pr){
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
    int time = 0;
```

```
        time = p[i].bt;
        p[i].ct = time;
        p[i].tat = p[i].ct;
        p[i].wt = p[i].tat - p[i].bt;
        avg_wt += p[i].wt;
        avg_tat = p[i].tat;
    }
    printf("\nPID\tBT\tPR\tCT\tTAT\tWT\n");
    for(i=0; i<n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].bt, p[i].pr, p[i].ct, p[i].tat,
            p[i].wt);
    }
    printf("\nAverage waiting time = %0.2f", avg_wt/n);
    printf("\nAverage Turnaround time = %0.2f\n", avg_tat/n);
    return 0;
}
```

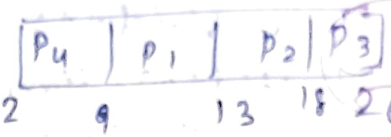
O/P.-

```
Enter no. of process: 4
Enter AT, BT and Pri for P1: 3 4 1
Enter AT, BT and Pri for P2: 4 5 3
Enter AT, BT and Pri for P3: 6 2 4
Enter AT, BT and Pri for P4: 2 7 4
```

PID	AT	BT	Pri	CT	TAT	WT
P1	3	4	1	13	10	6
P2	4	5	3	18	14	9
P3	6	2	4	20	14	12
P4	2	7	2	9	7	0

Average WT = 6.75
Avrge TAT = 11.25

gant chart -



Pscemptive

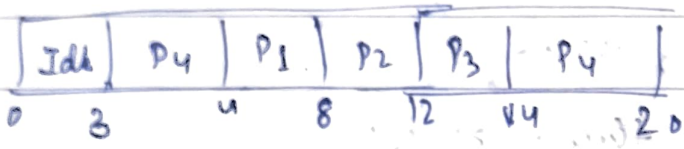
```
# include <stdio.h>
int main(){
  int n, i, time=0, completed=0;
  int at[20], bt[20], pr[0];
  int rt[20], ct[20], tat[20], wt[20];
  float avg_wt=0, avg_tat=0;

  printf("Enter ino.g processes:");
  scanf("%od", &n);
  for (i=0; i<n; i++){
    printf("Enter arrival time, BT, Pri for P%od", i+1);
    scanf("%od, %od %od", &at[i], &bt[i], &pr[i]);
    rt[i]=bt[i];
  }
while (completed<n){
  int idx=-1;
  int highest_pr=9999;
  for (i=0; i<n; i++){
    if (at[i]<=time && rt[i]>0){
      if (pr[i]<highest_pr){
        highest_pr=pr[i];
        idx=i;
      }
    }
  }
  if (idx!=-1){
    rt[idx]--;
    time++;
```

Average WT = 5.00

average TAT = 9.5

Gantt Chart:-



Week-4 (Round-Robin Scheduling)

⇒
```
#include <stdio.h>
int main(){
    int n, i, q;
    printf("Enter no of processes:");
    scanf("%od", &n);
    int bt[n], at[n];

    for (i=0; i<n; i++){
        printf("\n Process P%d \n", i+1);
        printf("Arrival time:");
        scanf("%d", &bt[i]);
    }
    printf("\n\nEnter Time quantum:");
    scanf("%od", &q);

    int rt[n], wt[n], tat[n], ct[n];
    int completed = 0, time = 0;
    for(i=0; i<n; i++)
        rt[i] = bt[i];
    printf("\n Gantt chart: \n|");
```

```
        if (at[i] <= t && rt[i] > 0) {
            done = 0;
            if (rt[i] <= q) {
                t += rt[i];
                rt[i] = 0;
            } else {
                t += q;
                rt[i] -= q;
            }
            printf("%d", t);
        }
    }
    if (done) break;
}
float total_wt = 0, total_tat = 0;
printf("\n\n Process \tAT\tBT\tCT\tWT\tTAT\n");
for (i = 0; i < n; i++) {
    printf("P%d \t%d\t%d \t%d \t%d \t%d \n",
           i+1, at[i], bt[i], ct[i], wt[i], tat[i]);
    total_wt += wt[i];
    total_tat += tat[i];
}
printf("\nAvg WT = %0.2f", total_wt/n);
printf("\nAvg TAT = %0.2f", total_tat/n);
return 0;
}
```

O/P: Enter no. of process: 4

Enter time quantum: 2

P1 AT BT : 3 6

P2 AT BT : 7 4

P3 AT BT : 1 5

P4 AT BT : 4 8

Gantt chart:

P3	P1	P3	P4	P1	P2	P3	P4	P1	P2	P4
0	3	5	7	9	11	13	14	16	18	20

Procss	AT	BT	CT	WT	TAT	
P_1	3	6	16	9	15	Average WT = 9.5
P_2	7	4	20	9	13	Average TAT = 15.25
P_3	1	5	14	8	13	
P_4	4	8	24	12	20	

Quantum time = 4.

Gantt Chart:- | P_3 | P_1 | P_4 | P_3 | P_2 | P_1 | P_4 |

0 5 9 13 14 18 20 24

Average WT = 9.5

Average TAT = 15.25

Quantum time = 6

Gantt chart: | P_3 | P_1 | P_4 | P_2 | P_4 |

0 6 12 18 22 24

Average WT = 6.5

Average TAT = 12.25

Lab 5 c-

Multilevel Queuing.

```
#include <stdio.h>

struct process{
    int id, at, bt, type;
    int ct, wt, tat;
};

void sort(struct process p[], int n){
    struct process temp;
    for (int i=0; i<n-1; i++){
        for (int j=i+1; j<n; j++){
            if (p[i].type > p[j].type ||
                (p[i].type == p[j].type && p[i].at > p[j].at)){
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}

int main(){
    int n;
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    struct process p[n];
    int gantt_p[100], gantt_t[100], q=0;

    for (int i=0; i<n; i++){
        p[i].id = i;
        printf("\n Process %d\n", i);
        printf("Enter arrival time: ");
```

Enter no. of processes: 4

Process 0

Enter arrival time: 2

Enter BT: 5

Enter type (0 = System, 1 = User): 1

Process 1

Enter AT: 1

Enter BT: 3

Enter type (0 = system, 1 = User): 0

Process 2

Enter arrival time: 3

Enter BT: 5

Enter type (0 = Sys, 1 = User): 0

Process 3

Enter AT: 4

Enter BT: 8

Enter type (0 = Sys, 1 = User): 1

ID	type	AT	BT	CT	WT	TAT
1	System	1	3	4	0	3
2	System	3	5	9	1	6
0	User	2	5	14	7	12
3	User	4	8	22	10	18

Average WT: 4.50

Average TAT: 9.75

gantt chart:

P_1	P_2	P_0	P_3

1 4 9 14 22

Lab 6:-

RMS:

```
#include <stdio.h>
#include <math.h>
#define MAX 10
typedef struct {
    int id;
    int bt; int period; int deadline; int ct; wt, tat;
} Process;
void swap (Process *a, Process *b){
    Process temp = *a;
    *a = *b;
    *b = temp;
}
int main (){
    int n;
    Process p[MAX];
    printf ("Enter no. of processes:");
    scanf ("%d", &n);
    printf ("\n Enter process details:\n");

for (int i=0; i<n; i++){
    p[i].id = i;
    printf ("\n Process %d:\n", i);
    printf ("%d", &p[i].period);
    printf ("\n Burst time :");
    scanf ("%d", &p[i]. ~~period~~ bt)
    printf ("\n Period:")
                scanf ("%d", &p[i].deadline
```

RMS scheduling:

ID	BT	Period	CT	WT	TAT
1	2	5	2	0	2
2	2	10	4	2	4
0	3	20	7	4	7

Deadline check:

P_1 met deadline

P_2 met deadline

P_0 met deadline

EDF

```
#include <stdio.h>
#define MAX 10
typedef struct{
    int id, bt, deadline, period;
    int ct, wt, tat;
}Process;

void swap(Process *a, Process *b){
    Process temp = *a;
    *a = *b;
    *b = temp;
}

int main(){
    int n;
    Process p[MAX];
```

```
int time = 0;
printf("\n Gantt Chart :\n");
for (int i = 0; i < n; i++) {
    printf("P%d ", p[i].id);
}

time = 0;
for (int i = 0; i < n; i++) {
    time += p[i].bt;
    p[i].ct = time;
    p[i].tat = p[i].ct;
    p[i].wt = p[i].tat -
}
for (int i = 0; i < n; i++) {
    printf("%d \t%d \t%d \t%d \t%d \t%d \n",
        p[i].id, p[i].bt, p[i].deadline, p[i].ct, p[i].wt,
        p[i].tat);
}
return 0;
}.
```

Q/O:- Enter no. of process : 3

Enter process details:

P_0 : BT: 2
Deadline: 6
Period: 4

P_1 : BT: 4
Deadline: 7
Period: 2

P_2 : BT: 6
Deadline: 8
Period: 6

CPU Utilization = 3.50

Task set is NOT SCHEDULABLE under EDF

Gantt Chart:

| P_0 | P_1 | P_2 |
0 2 6 12

ID	BT	Deadline	CT	WT	TAT
0	2	6	2	0	2
1	4	7	6	2	6
2	6	8	12	6	12

Dining Philosopher :-

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5
sem_t forks[N];
sem_t room;

void *philosopher(void *num){
    int id = *(int*)num;
    while(1){
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        sem_wait(&room);

        sem_wait(&forks[id]);
        printf("Philosopher %d picked up left fork %d.\n",
                id, id);
        sem_wait(&forks[(id+1)%N]);
        printf("Philosopher %d picked up right fork %d.\n",
            id, (id+1)%N);

        printf("Philosopher %d put down forks %d and
                %d.\n", id, id, (id+1)%N);
        sem_post(&room);
    }
}
```

Output:-

Tracing.

Produced : 0 at buff[0]. [0, -, -] in - 1, out - 2

Produced : 1 at buff[1] [0, 1, -] in = 2, out - 1

Consumed : 0 from buff[0] [-, 1, -] in - 1, out 2

Produced : 2 at buff[2]. [-, 1, 2] in 2, out - 1

Consumed : 1 from buff[1]. [-, -, 2]

Produced : 3 at buffer[0]. [3, -, 2]

Consumed : 2 from buffer[2]. [3, -, -]

Produced : 4 at buffer[1] [3, 4, -]

Consumed : 3 from buffer[0] [-, 4, -]

Produced : 5 at buffer[2] [-, 4, 5]

Consumed : 4 from buffer[1]. [-, -, 5]

Produced 6 : at buffer[0]. [6, -, 5]

consumed : 5 from buffer[0] [6, -, -]

Produced : 7 at buff[1]. [6, 7, -]

Consumed 6 from buff[0] [-, 7, -]

Consumed : 7 from buff[1] [-, -, -].

Bounded Buffer :-

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 3
#define ITEMS 8

int buffer[SIZE];
int in=0, out=0;
sem_t empty, full, mutex;

void *producer(void *arg){
  for(int i=0; i<ITEMS; i++){
      sem_wait(&empty);
      sem_wait(&mutex);

      buffer[in]=i;
      printf("Produced: %d at buffer[%d]\n", i, in);

      in=(in+1)%SIZE;
      sem_post(&mutex);
      sem_post(&full);
```

Banker's Algorithm:

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 10
int main(){
    int n, m;
    int Allocation[MAX][MAX], Max[MAX][MAX], Need[MAX][MAX];
    int Available[MAX], Work[MAX];
    bool Finish[MAX];
    int SafeSequence[MAX];

    printf("Enter no. of processes: ");
    scanf("%d", &n);
    printf("Enter no. of resources: ");
    scanf("%d", &m);

    printf("Enter Allocation Matrix:\n");
    for(int i=0; i<n; i++){
        for(int j=0; j<m; j++){
            scanf("%d", &Allocation[i][j]);
        }
    }
    printf("Enter Maximum Demand Matrix:\n");
    for(int i=0; i<n; i++){
        for(int j=0; j<m; j++){
            scanf("%d", &Max[i][j]);
        }
    }
}
```

o/p: Enter no. of processes: 3 5

Enter no. of resources: 4 3

Enter Allocation Matrix: 0 1 0

2 1 3 2 0 0

1 1 0 3 0 2

2 1 0 2 1 1

3 4 5 0 0 2

Enter Maximum Demand Matrix: 7 5 3

2 3 4 3 2 2

5 6 7 9 0 2

2 1 3 2 2 2

0 1 0 4 3 3

Enter Available Resources:

1 2 3 4 3 3 2

System is not in safe state => $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$

Resource Request Algorithm:

```
if (choice == 2) {
    int p, Request[MAX];
    printf("Enter process no requesting resources: ");
    scanf("%d", &p);
    printf("Enter Request Vector:\n");
    for(int j=0; j<m; j++) {
        scanf("%d", &Request[j]);
    }
    for(int j=0; j<m; j++){
        if(Request[j] > Need[p][j]) {
            printf("Error: Request exceeds Need.\n");
            return 0;
        }
    }
}
```

```
for (int j = 0; j < m; j++) {
    if (Request[j] > Available[j]) {
        printf("Resources not available. Process must wait!\n");
        return 0;
    }
}
for (int j = 0; j < m; j++) {
    Available[j] -= Request[j];
    Allocation[p][j] += Request[j];
    Need[p][j] -= Request[j];
}
}
```

O/P:-

1. Safety Algorithm
2. Resource Request Algorithm

Enter your choice : 2

Enter no. of processes : 5

Enter no. of resources : 3

Enter Allocation Matrix:	Enter Maximum Demand Matrix:
0 1 0	7 5 3
2 0 0	3 2 2
3 0 2	9 0 2
2 1 1	2 2 2
0 0 2	4 3 3

Enter Available Resources:

3 3 2

Enter process number requesting resources: 1

Enter Request Vector: 1 0 2

Request can be granted. System remains safe.

Safe sequence: $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$

Deadlock Detection:-

```
#include <stdio.h>

int main(){
    int n, m, i, j, k;
    int A[10][10], R[10][10];
    int Av[10], W[10], F[10];
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    printf("Enter no. of resources: ");
    for(j=0; j<m; scanf("%d", &m);
    printf("Enter Allocation Matrix:\n");
    for(i=0; i<n; i++)
        for(j=0; j<m; j++)
            scanf("%d", &A[i][j]);
    printf("Enter Request Matrix:\n");
    for(i=0; i<n; i++)
        for(j=0; j<m; j++)
            scanf("%d", &R[i][j]);

    printf("Enter Available Resources:\n");
    for(j=0; j<m; j++)
        scanf("%d", &Av[j]);
    for(j=0; j<m; j++)
        W[j] = Av[j];

    for(i=0; i<n; i++){
        int zero = 1;
```

o/p- Enter no. of process: 4

Enter no. of resources: 3

Enter Allocation matrix:

1 0 2

2 1 1

1 0 3

1 2 2

Enter Request Matrix:

0 0 1

1 0 2

0 0 0

3 3 0

Enter Available resources

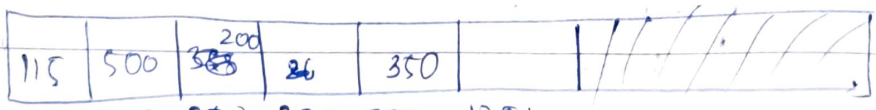
0 0 0

deadlocked Process: P_3.

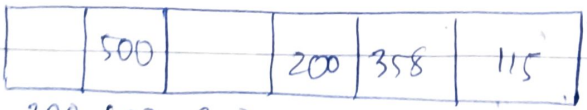
Partitions:- 300K, 600K, 360K, 200K, 750K, 125K.

Process size:- 115, 500, 358, 200, 375

firstfit:-



Bestfit:- 375 not fit



=> 375 not allocated.

worstfit:-

358 and 375 not allocated

Memory Allocation Technique

#

```
#include <stdio.h>
int main(){
  int b[10], p[10], used[10], nb, np;
  int i, j, k, x, max, min;

  printf("Enter no. of memory block: ");
  scanf("%od", &nb)

  printf("Enter sizes of %od memory block: \n", nb);
  for(i=0; i<nb; i++)
     scanf("%od", &b[i]);
  printf("Enter number of processes: ");
  scanf("%od", &np);
  printf("Enter sizes of %od processes: \n", np);
  for(i=0; i<np; i++)
     scanf("%od", &p[i]);

  for(x=1; x<=3; x++){
     for(i=0; i<nb; i++)
        used[i]=0;

  if (x==1) printf("\n... First Fit ...\n");
  if (x==2) printf("\n... Best Fit ...\n");
  if (x==3) printf("\n... worst Fit ...\n");
```

o/p:- Enter no. of memory block : 5

Enter size of memory blocks:

400

700

200

300

600

Enter no. of processes: 4.

Enter size of processes:

212 512 312 526

FIRST FIT ALLOCATION

Process NO	Process Size	Block NO
1	212	1
2	512	2
3	312	5
4	526	Not allocated

BEST FIT ALLOCATION

Process NO	Process Size	Block NO
1	212	4
2	512	5
3	312	3
4	526	2

WORST FIT ALLOCATION:

1	212	2
2	512	5
3	312	1
4	526	Not Allocated.

Page replacement Algorithm:-

```
#include <stdio.h>
#define MAX 50
void displayFrames(int frame[], int f) {
    int i;
    for(i=0; i<f; i++){
        if (frame[i] != -1)
            printf(" %d", frame[i]);
        else
            printf(" -");
    }
    printf("\n");
}
void FIFO (int page[]; int n, int f) {
    int frame[MAX], i, j, k=0, flag, faults = 0;
    for (i=0; i<f; )++) {
        frame[i] = -1;
    printf("\n FIFO Page Replacement:\n");
    for(i=0; i<n; ++){
        flag=0;
        for (j=0; j<f; j++){
            if (frame[j] == page[i]) {
                flag = 1;
                break;
            }
        }
        if (flag == 0); {
            frame[k] = page[i];
            k= (k+1)%f;
            faults ++;
        }
```

```
        printf("Page %od → ", pages[i]);
        displayFrames(frames, f);
    }
    printf("Total pages Faults = %d\n", faults);
}

int main(){
    int pages[MAX], n, f, i;
    printf("Enter no. of pages:");
    scanf("%d", &n);
    printf("Enter page reference string:\n");
    for(i=0; i<n; i++){
        scanf("%d", &pages[i]);
    printf("Enter no. of frames:");
    scanf("%d", &f);
    FIFO(pages, n, f);
    LRU(pages, n, f)
    Optimal(pages, n, f);
    return 0;
};
```

O/P:- Enter no. of pages: 20
Enter the page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Enter no. of frames: 3
Total no. of faults for FIFO: 15
Total no. of faults for LRU: 12
Total no. of frams for Optimal: 9

=> Optimal performs well among Three

FIFO

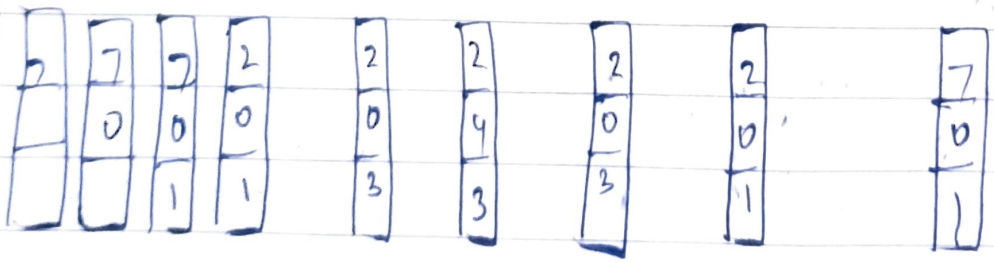
* 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0

* LRU:-

7 0 1 2 0 3 0 4 2 3 0 3 2 1 0 1 7 0 1

* Optimal:-

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



* The best suited is optimal page Replacement